CYBERSECURITY
PRACTICAL REPORT

Network Protocol Investigation

A Wireshark-based study of captured traffic in a Kali Linux environment

STUDENT	Neralla Sairam Suraj
ROLL NUMBER	STU-B33F7C
COURSE / GROUP	Cyber Security / G17
PLATFORM	SURE ProEd
SUBMISSION DATE	22 October 2026

PROJECT FOCUS

Read captured packets as evidence: identify the protocol, understand the role of its headers, and distinguish directly visible fields from background explanation. The screenshots retained in this report come from the supplied Wireshark evidence.

01 | Investigation brief

Purpose

This practical examines how common network services appear in packet captures. Rather than treating protocols as isolated definitions, the analysis follows the evidence across name resolution, connectivity checks, transport setup, time synchronisation, address configuration and encrypted web traffic.

Questions explored

- Which fields help match a DNS request to its response?
- How do ICMPv6 echo messages and neighbour discovery support IPv6 connectivity?
- What sequence of TCP flags indicates the start of a connection?
- Which UDP ports identify DNS, NTP and DHCP in the supplied examples?
- What can be learned from a TLS 1.3 Client Hello without decrypting application data?

Capture context

The supplied evidence identifies Wireshark 4.4.7 running on Kali Linux in a VMware Fusion environment, capturing on **eth0**. The main connectivity examples use IPv6. The archive also includes a separate DHCP-over-IPv4 example; this is reported as a separate capture context rather than treated as contradictory evidence.

Evidence handling

Only fields visible in the screenshots are described as observed results. Protocol behaviour is explained separately, and encrypted TLS application contents are not inferred from the capture.

OSI REFERENCE | THE SEVEN LAYERS

A layered view of how data moves from an application to the physical network.

7	**APPLICATION** Network services used by end-user applications.	HTTP, HTTPS, DNS
6	**PRESENTATION** Data format, character encoding, compression and encryption.	TLS/SSL, JPEG, UTF-8
5	**SESSION** Establishes, manages and ends communication sessions.	Session control / RPC
4	**TRANSPORT** End-to-end delivery, ports, reliability and segmentation.	TCP, UDP | Segment / Datagram
3	**NETWORK** Logical addressing and routing between networks.	IP, ICMP | Packet
2	**DATA LINK** Local-link frames, MAC addressing and error detection.	Ethernet, Wi-Fi | Frame
1	**PHYSICAL** Transmits raw bits over a physical or radio medium.	Cables, radio | Bits

Note: Wireshark captures expose protocol details at multiple layers; the exact fields depend on the packet.

02 | Protocol map

Protocol	Observed clue	Interpretation
DNS	Name query and reply; UDP port 53	Maps a host name to address records; transaction ID and record type help correlate messages.
ICMPv6	Echo Request / Echo Reply; types 128 / 129	Tests IPv6 reachability. Neighbor Solicitation / Advertisement (135 / 136) discover local-link neighbours.
TCP	SYN → SYN-ACK → ACK; ports 44034 and 443	Three-way handshake establishes the transport connection used for HTTPS.
NTP	Frame 39; UDP 37534 → 123; 48-byte payload	Client request for time synchronisation.
DHCP	Frame 49; 0.0.0.0 → broadcast; UDP 68 → 67	Request for IPv4 configuration; requested address shown as 10.172.213.17.
TLS 1.3	Frame 8; Client Hello to www.microsoft.com	Handshake metadata includes SNI, 30 cipher suites and ALPN; application data remains encrypted.

Layer perspective

The capture view combines several layers in a single frame. Ethernet provides local-link context; IPv6 or IPv4 carries network addressing; TCP or UDP provides transport; and DNS, NTP, DHCP or TLS-related messages provide service-level meaning. The OSI model is a useful guide, although real protocol boundaries do not always map perfectly to a single layer.

03 | Environment and workflow

Item	Recorded configuration
Operating environment	macOS host; VMware Fusion virtual machine
Guest system	Kali Linux
Capture application	Wireshark 4.4.7
Interface	eth0
VM MAC address	00:0c:29:c5:97:80
Global IPv6 address	2409:40e6:435:6928:2ecd:10e4:7158:11a5
Link-local IPv6 address	fe80::20c:29ff:fec5:9780
Router next hop	fe80::824:45ff:fe96:147f
DNS server	2409:40e6:435:6928::91

Workflow used

1. Start a capture on eth0.
2. Generate traffic with **ping -c 6 google.com** and **curl -I https://www.microsoft.com**.
3. Apply display filters to isolate the protocol under review.
4. Expand packet details and compare the visible header fields.
5. Record observations and explain their role without assuming hidden payload values.

Tools referenced

Wireshark, Kali Linux, ping and curl. The supplied archive also contains separate NTP, DHCP and TLS 1.3 evidence.

04 | Packet evidence

Transport connection and service traffic

```
▾ User Datagram Protocol, Src Port: 37534, Dst Port: 123
    Source Port: 37534
    Destination Port: 123
    Length: 56
    Checksum: 0x5c82 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 5]
    [Stream Packet Number: 1]
  ▾ [Timestamps]
      [Time since first frame: 0.000000000 seconds]
      [Time since previous frame: 0.000000000 seconds]
    UDP payload (48 bytes)
▾ Network Time Protocol (NTP Version 4, client)
  ▸ Flags: 0x23, Leap Indicator: no warning, Version number: NTP Version 4, Mode: client
    [Response In: 40]
    Peer Clock Stratum: unspecified or invalid (0)
    Peer Polling Interval: 0 (1 seconds)
    Peer Clock Precision: 0 (1.000000000 seconds)
    Root Delay: 0.000000 seconds
    Root Dispersion: 0.000000 seconds
    Reference ID: NULL
    Reference Timestamp: NULL
    Origin Timestamp: NULL
    Receive Timestamp: NULL
    Transmit Timestamp: Feb 16, 1978 11:55:27.856571563 UTC
```

Figure 1. Captured TCP handshake evidence from the supplied Wireshark screenshots.

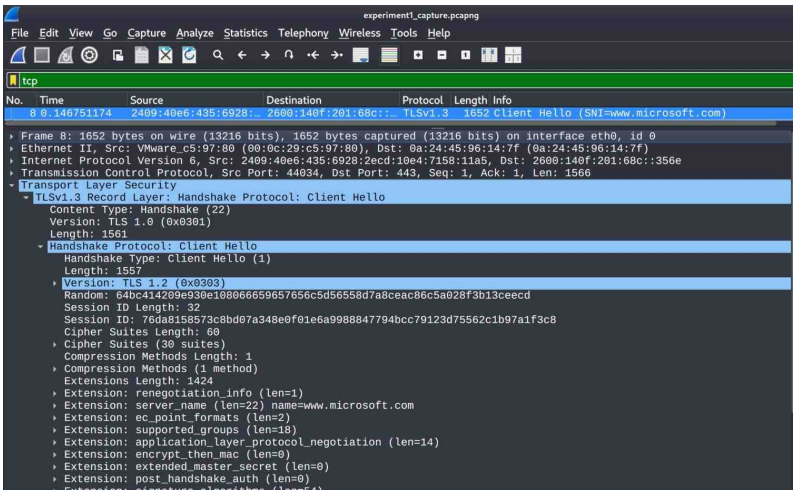


Figure 2. DNS and UDP packet details retained from the original evidence.

Packet evidence (continued)

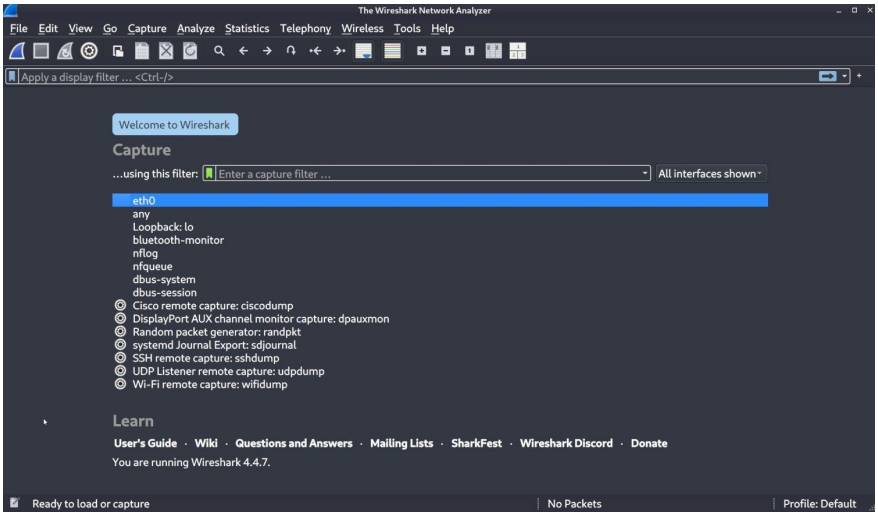


Figure 3. NTP Version 4 client packet: UDP destination port 123.

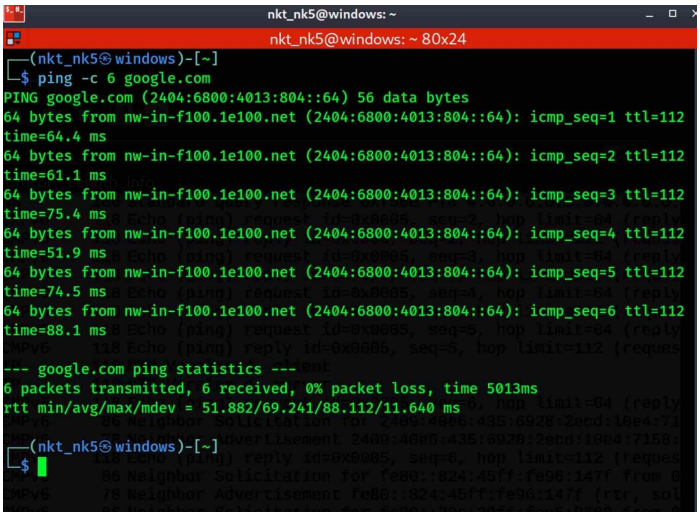


Figure 4. DHCP Request capture showing IPv4 broadcast and UDP ports 68 → 67.

05 | Secure web traffic

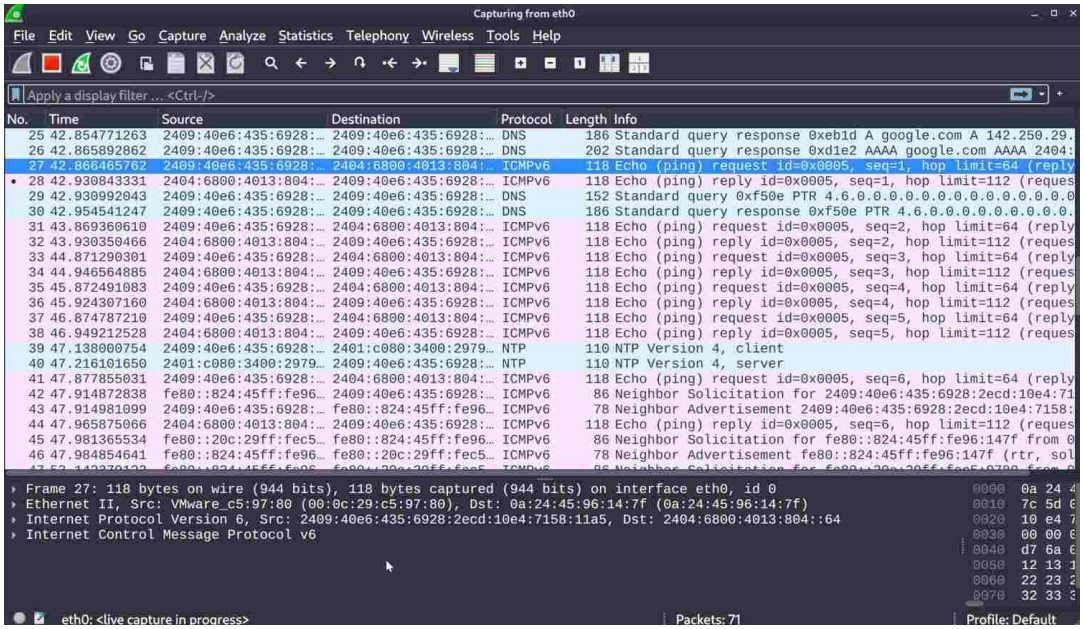


Figure 5. TLS 1.3 Client Hello evidence; visible handshake metadata only.

Reading the TLS evidence

The Client Hello exposes negotiation information such as the server name indication, supported cipher suites and ALPN extension. These fields describe the initial security negotiation. They do not reveal the content of the encrypted web session, so no claim about the page contents is made here.

Display filters used for focused review

dns — isolate DNS messages **icmpv6** — inspect IPv6 control messages
tcp — review connection setup **udp** — inspect UDP-based services
ntp — locate time synchronisation **dhcp** — locate DHCP exchanges
tls — inspect TLS handshake records

06 | Findings and conclusion

Main findings

• DNS, NTP and DHCP use UDP in the supplied examples; the destination ports help identify the service involved.

• The TCP example shows the standard SYN, SYN-ACK and ACK sequence before HTTPS data exchange.

• ICMPv6 supports both reachability tests and local IPv6 neighbour discovery.

• The DHCP example is an IPv4 broadcast request, while the main connectivity evidence includes IPv6 traffic.

• TLS 1.3 reveals useful handshake metadata, but encrypted application payloads should not be interpreted without authorised decryption material.

Conclusion

The exercise demonstrates how packet analysis links protocol theory to observable network behaviour. By combining packet filters, header inspection and a disciplined distinction between visible evidence and protocol background, Wireshark can help explain connectivity, configuration and secure-session setup. The supplied captures cover several layers and services, and together they provide a practical introduction to network troubleshooting and cybersecurity investigation.

Limitations

This report is based on the supplied screenshots and packet evidence, not a newly collected live capture. Some fields may be unavailable or truncated in screenshots; those values are not reconstructed. The TLS application payload remains encrypted.

END OF REPORT • MINI PROJECT 02